BOOTCAMP QI TECH 2026 · AULA 3 (26/09) · MATERIAL DO PARTICIPANTE

RFC preenchido — o serviço de biblioteca

O modelo de RFC da Aula 2 preenchido de ponta a ponta para um serviço de biblioteca: cadastro de livros pelo ISBN, lotação de estante e empréstimo.

RFC — Biblioteca: cadastro de livros pelo ISBN, lotação de estante e empréstimo

TIME	Time do serviço de biblioteca
DATA	26/09/2026
VERSÃO	1.0

Contextualização

Entendendo o problema

A biblioteca cadastra autores, estantes, leitores e livros, põe cada livro numa estante e controla quais livros estão disponíveis e com qual leitor está cada livro emprestado. Quem chama o serviço são outros sistemas, identificados por um token interno.

O serviço precisa sustentar três garantias:

- **Lotação da estante:** uma estante nunca guarda mais livros que a capacidade dela, mesmo com dois pedidos chegando ao mesmo tempo para a última vaga. Se falhar, a estante física transborda e o inventário deixa de bater com o que está nas prateleiras.
- **Cadastro único:** uma edição é cadastrada uma vez só; repetir o cadastro do mesmo ISBN não cria um segundo livro. Se falhar, o mesmo livro aparece duas vezes, com títulos ligeiramente diferentes.
- **Status do livro:** um livro só é emprestado se está disponível, e só é devolvido se está emprestado.

Título, ano e número de páginas não são digitados por ninguém: vêm de um catálogo de ISBN externo, que pode estar fora do ar.

Fora do escopo desta entrega: prazo e histórico de empréstimos, editar ou apagar autor, estante, leitor ou livro, e mudar a capacidade de uma estante

depois de criada.

Explicando a solução de forma macro

São quatro entidades — `author`, `shelf`, `member` e `book` — e um catálogo de ISBN de fora. O status do livro mora em tabela própria (`book_status`), e cada cadastro, empréstimo e devolução grava um evento em `book_status_event`, na mesma transação. O cliente cadastra o livro com `isbn` e `author_key`, e o serviço busca no catálogo título, ano e páginas. Cada requisição é uma transação, confirmada pela regra de negócio só no fim. Duas travas seguram a concorrência: a da linha da estante, da contagem até a gravação, garante a lotação; a da linha do livro, no empréstimo e na devolução, garante o status. O CPF de autor e de leitor é obrigatório e único dentro de cada tabela: a mesma pessoa pode ser autora e leitora com o mesmo CPF. Alternativas descartadas:

- **Contar livros sem travar a estante:** duas requisições contam 9 de 10 e ambas gravam.
- **Travar a estante antes do catálogo:** a trava duraria a chamada HTTP, até 5 s, e pararia a estante.
- **Pedir título, ano e páginas ao cliente:** abre erro de digitação e aceita ISBN que o catálogo recusaria.
- **Barrar o ISBN repetido só pelo `UNIQUE`:** chamaria o catálogo para um pedido que será recusado.
- **Tabela `loan` com histórico:** hoje só se pergunta com quem está o livro, e `book.member_id` responde; o evento de status não guarda o leitor.

Implementação

Rotas

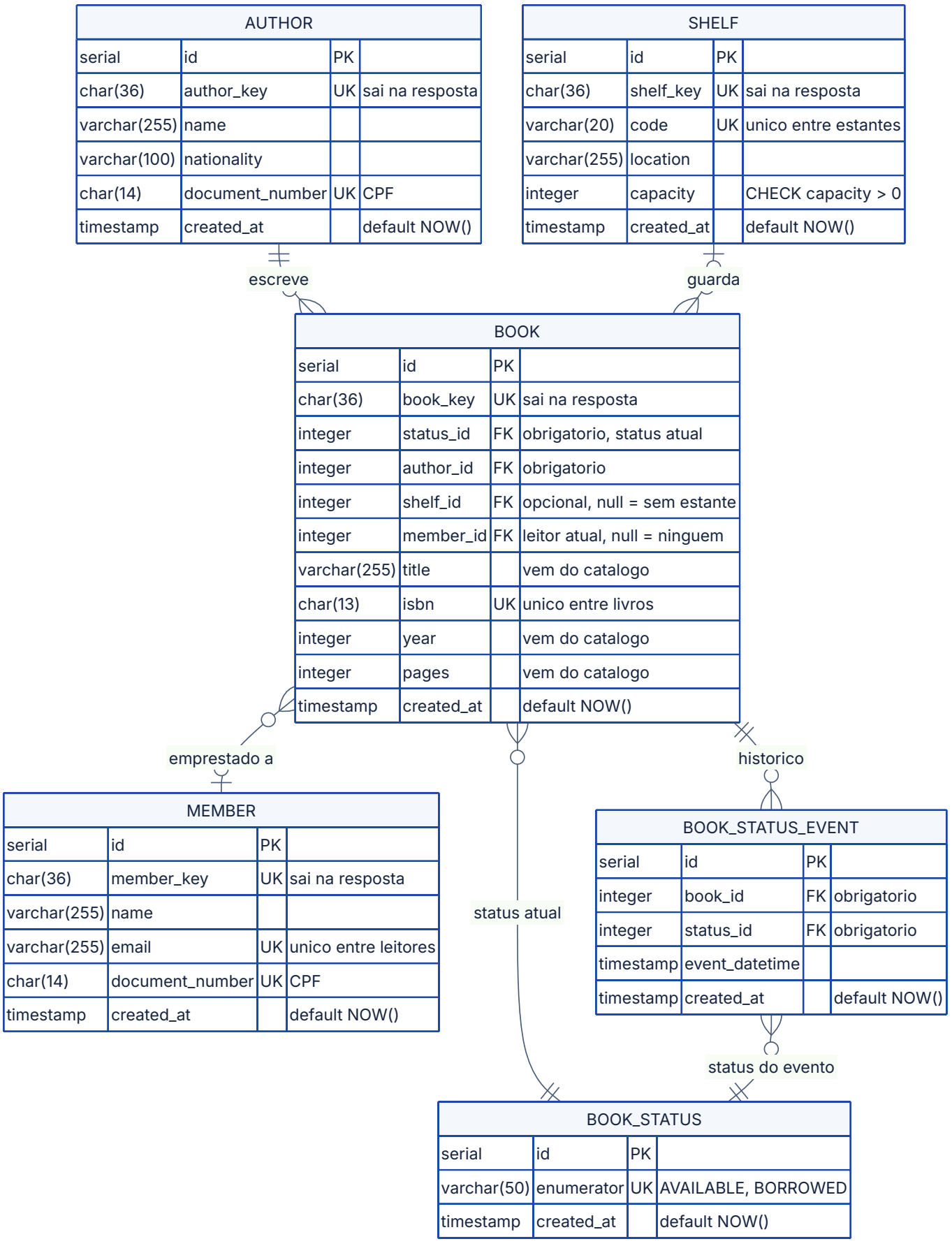
MÉTODO	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`GET`	`/`	Diz qual serviço é este. Aberta, sem token	—	`200`
`GET`	`/health_check`	Diz se o serviço está de pé. Aberta, sem token	—	`204`
`POST`	`/author`	Cadastra um autor. Repetir não cria outro: `UNIQUE(document_number)`	`name`, `nationality`, `document_number` (CPF `000.000.000-00`)	`201` com `author_key`; `400` `QIT000001` corpo fora do formato, incluindo CPF ausente ou fora da máscara; `409` `QIT001024` CPF já cadastrado; `422` `QIT001023` CPF com dígitos verificadores que não batem
`GET`	`/author/{author_key}`	Devolve um autor, com `document_number`	`author_key` no caminho	`200`; `404` `QIT001012` autor inexistente

MÉTODO	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`GET`	`/authors`	Lista autores, paginado	`limit` (padrão 10, teto 100), `page` (padrão 0)	`200` com `data`, `limit`, `page`, `is_last_page`; `400` `QIT000001` parâmetro fora do formato ou desconhecido
`POST`	`/shelf`	Cadastra uma estante. Repetir não cria outra: o `code` é único	`code`, `location`, `capacity` (inteiro, mínimo 1)	`201` com `shelf_key`; `400` `QIT000001` corpo fora do formato, incluindo `capacity` menor que 1; `409` `QIT001014` `code` já cadastrado
`GET`	`/shelf/{shelf_key}`	Devolve uma estante	`shelf_key` no caminho	`200`; `404` `QIT001013` estante inexistente
`GET`	`/shelves`	Lista estantes, paginado	`limit`, `page`	`200` com a página; `400` `QIT000001`
`POST`	`/member`	Cadastra um leitor. Repetir não cria outro: o `email` é único	`name`, `email`, `document_number` (CPF `000.000.000-00`)	`201` com `member_key`; `400` `QIT000001` corpo fora do formato, incluindo CPF ausente ou fora da máscara; `409` `QIT001022` `email` já cadastrado, `QIT001024` CPF já cadastrado; `422` `QIT001023` CPF com dígitos verificadores que não batem
`GET`	`/member/{member_key}`	Devolve um leitor	`member_key` no caminho	`200`; `404` `QIT001021` leitor inexistente
`GET`	`/members`	Lista leitores, paginado	`limit`, `page`	`200` com a página; `400` `QIT000001`
`POST`	`/book`	Cadastra um livro a partir do ISBN, com título, ano e páginas vindos do catálogo. Repetir não cria outro: `UNIQUE(isbn)`	`isbn` (13 dígitos), `author_key`, `shelf_key` (opcional)	`201` com `book_key` e `status`; `400` `QIT000001` corpo fora do formato; `404` `QIT001012` autor inexistente, `QIT001013` estante inexistente, `QIT001018` o catálogo não conhece o ISBN; `409` `QIT001016` ISBN já cadastrado, `QIT001017` estante lotada; `502` `QIT001019` catálogo sem resposta, com outro status ou sem corpo JSON
`GET`	`/book/{book_key}`	Devolve um livro, com `author_key`, `shelf_key` (ou `null`), `status`, `member_key` (ou `null`) e `status_events` (o histórico, do cadastro em diante, em ordem cronológica, cada item com `status` e `event_datetime`)	`book_key` no caminho	`200`; `404` `QIT001015` livro inexistente
`GET`	`/books`	Lista livros, paginado, com filtro opcional por estante; a página vem sem `status_events`	`limit`, `page`, `shelf_key` (opcional)	`200` com a página; `400` `QIT000001`; `404` `QIT001013` filtro por estante inexistente

MÉTODO	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`PUT`	`/book/{book_key}/shelf`	Move o livro para a estante do corpo. Pedir a estante onde o livro já está responde `409`, antes de conferir lotação	`shelf_key`	`200` com `book_key` e `status`; `400` `QIT000001`; `404` `QIT001015` livro inexistente, `QIT001013` estante inexistente; `409` `QIT001025` o livro já está nessa estante, `QIT001017` estante lotada
`PUT`	`/book/{book_key}/borrow`	`AVAILABLE` → `BORROWED`, com o leitor do corpo gravado no livro. Repetir não muda o livro e responde `409`	`book_key` no caminho, `member_key` no corpo	`200` com `book_key` e `status`; `400` `QIT000001` corpo sem `member_key`; `404` `QIT001021` leitor inexistente, `QIT001015` livro inexistente; `409` `QIT001020` o livro não está `AVAILABLE`
`PUT`	`/book/{book_key}/return`	`BORROWED` → `AVAILABLE`, e o livro fica sem leitor (`member_key` `null`). Repetir não muda o livro e responde `409`	`book_key` no caminho	`200` com `book_key` e `status`; `404` `QIT001015`; `409` `QIT001020` o livro não está `BORROWED`

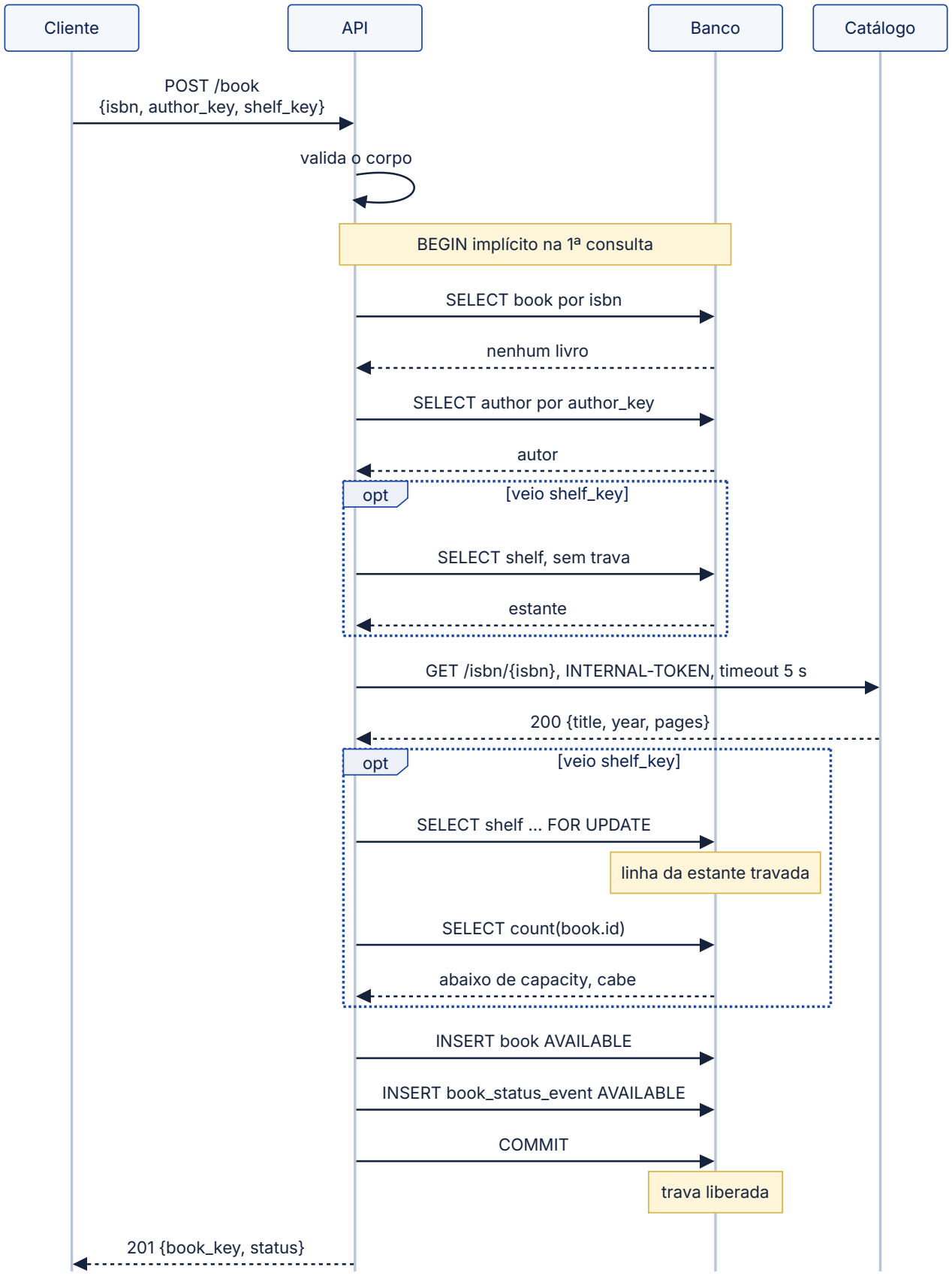
Todas as rotas, menos `/` e `/health_check`, exigem o cabeçalho `INTERNAL-TOKEN`; sem ele ou com valor errado, a resposta é `403` `QIT000002`.

Banco de Dados (Somente diagrama)



Fluxos

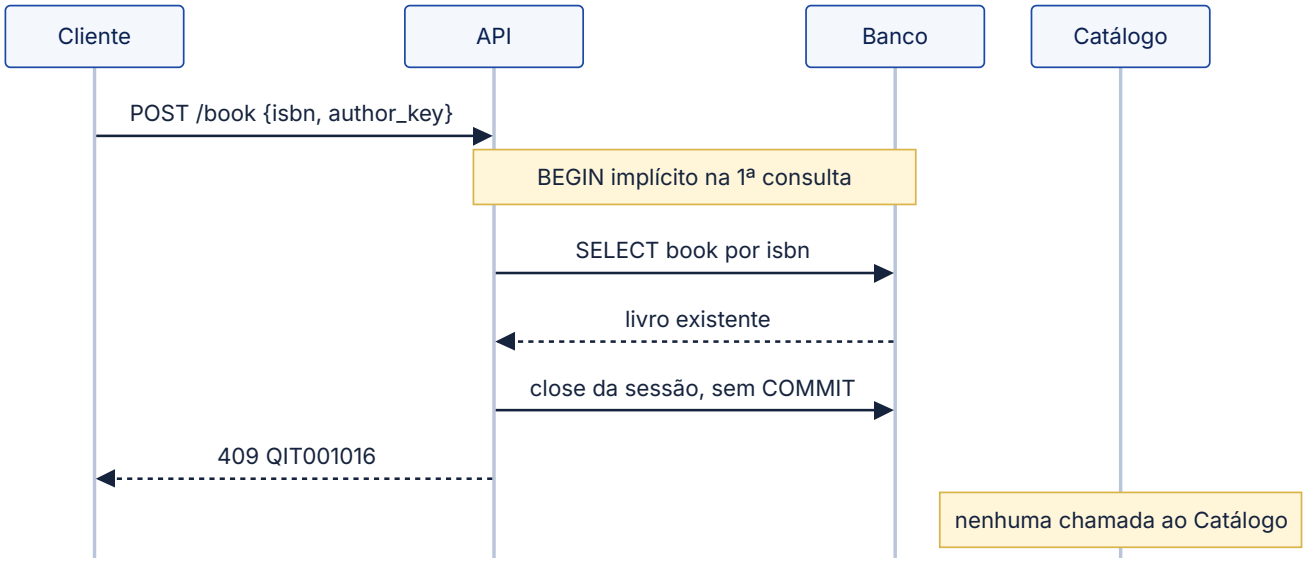
Cadastrar um livro — caminho feliz



1. O corpo é validado: `isbn` com 13 dígitos e `author_key` obrigatórios, `shelf_key` opcional. Fora do formato, `400 QIT000001` antes de qualquer consulta.

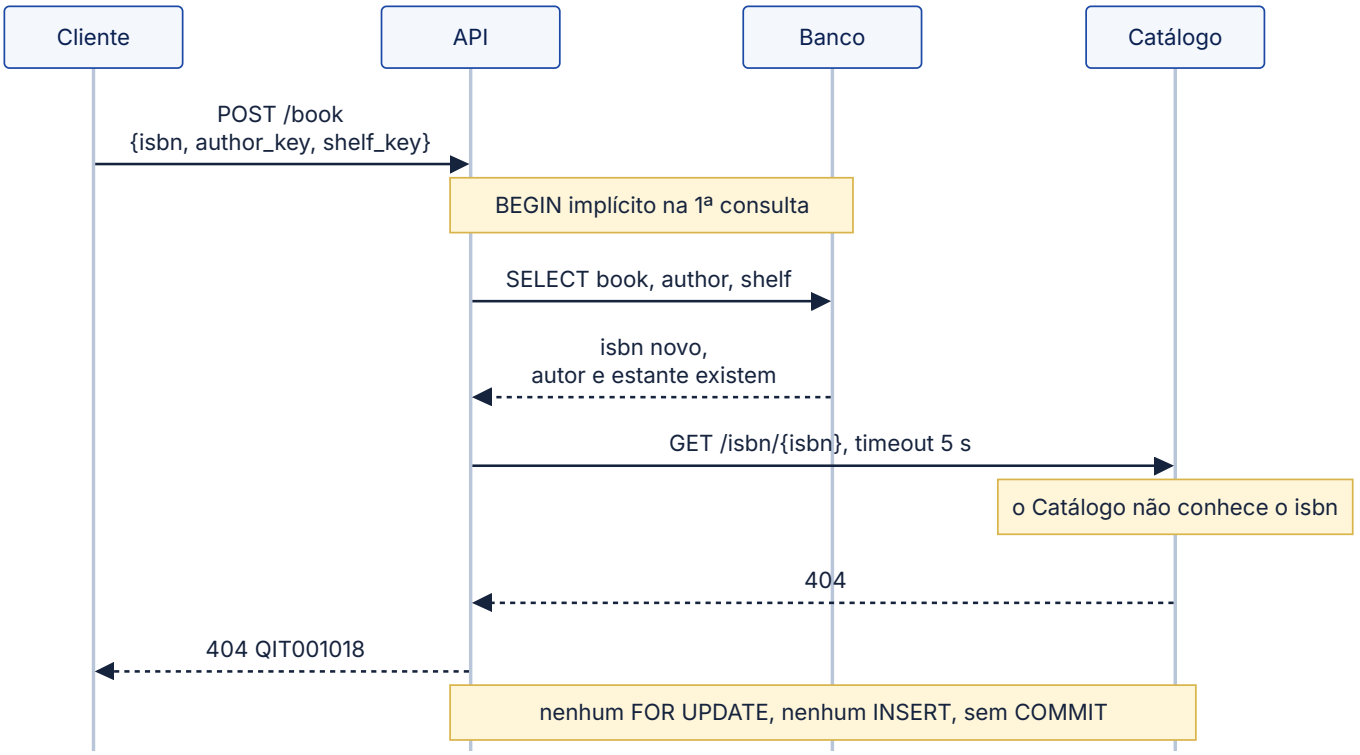
2. Procura um livro com aquele `isbn` no banco. Não há.
3. Procura o autor pela `author_key` e, se veio `shelf_key`, a estante. Sem autor, `404 QIT001012`; sem estante, `404 QIT001013`. Nenhuma dessas consultas trava linha.
4. Chama o catálogo: `GET /isbn/{isbn}`, com o próprio token interno e timeout de 5 segundos. O catálogo responde `200` com `title`, `year` e `pages`.
5. Se o livro vai para uma estante, trava a linha da estante (`SELECT … FOR UPDATE`), conta os livros que já estão nela e compara com `capacity`. Cabe.
6. Grava o livro com o status `AVAILABLE` em `status_id`, uma `book_key` nova e os dados do catálogo, e grava o primeiro evento do livro, `AVAILABLE`, em `book_status_event`.
7. Confirma a transação, o que libera a trava da estante. Responde `201` com a `book_key` e o `status`.

Cadastrar um livro — falha: ISBN já cadastrado



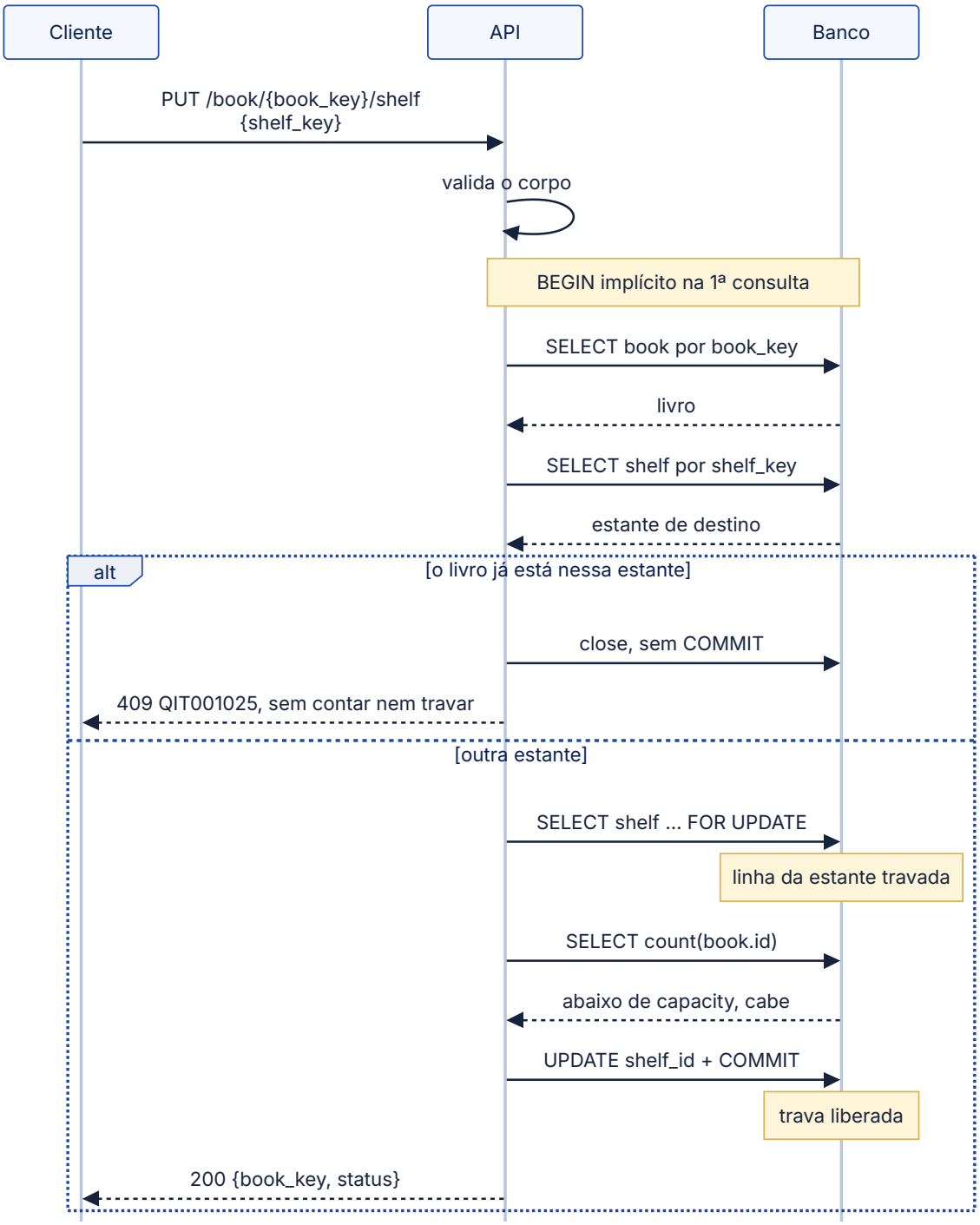
1. No passo 2, o livro com aquele `isbn` existe.
2. A API responde `409 QIT001016` ali mesmo. O catálogo não recebe nenhuma chamada, e nada é gravado.

Cadastrar um livro — falha: o catálogo não ajuda



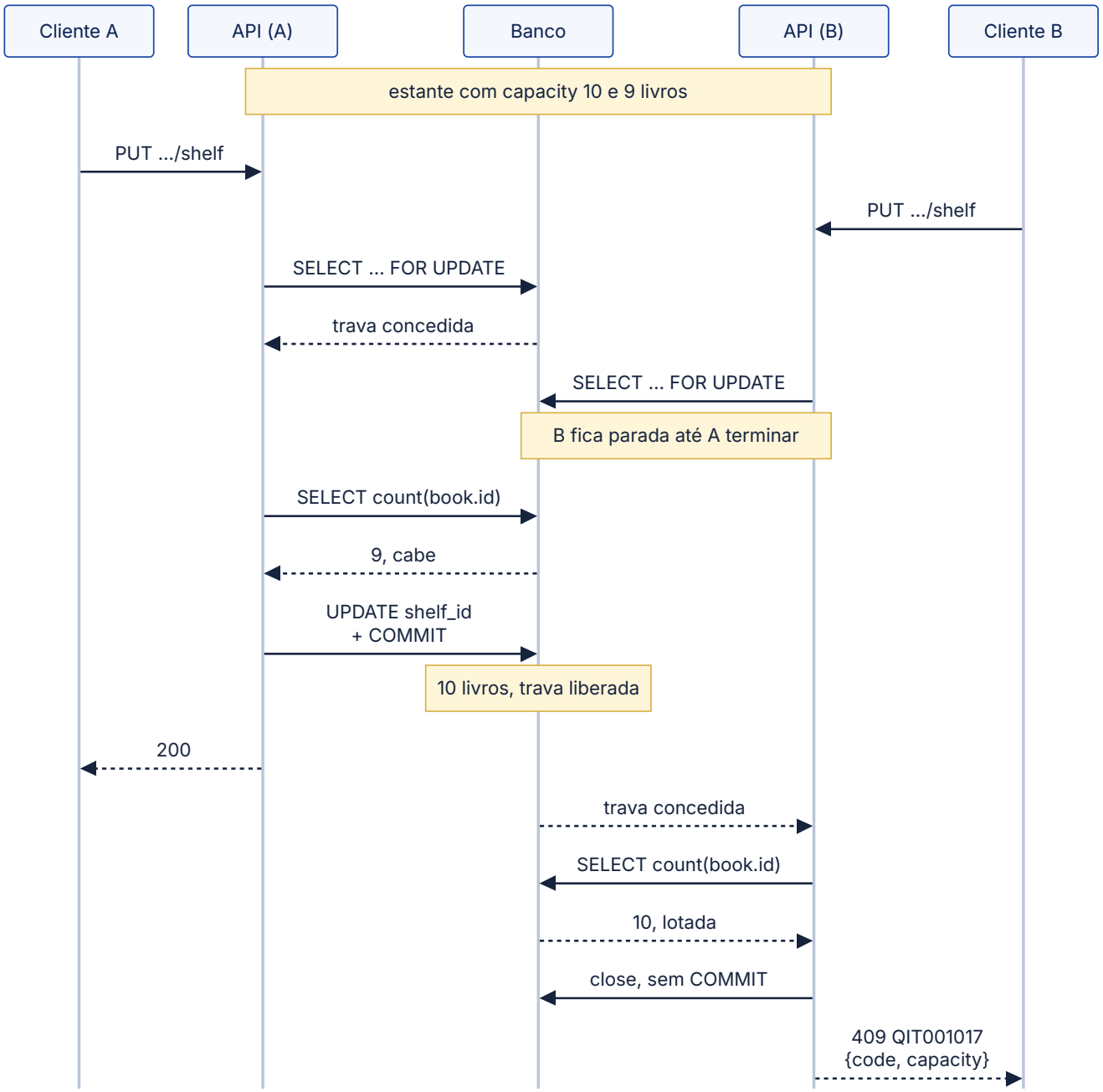
1. O catálogo responde `404`: a API responde `404 QIT001018`, código diferente do `QIT001015` (livro inexistente nesta API) para que o cliente consiga separar os dois casos.
2. O catálogo responde qualquer outro status ou uma resposta sem corpo JSON: `502 QIT001019`.
3. A conexão cai ou o timeout de 5 segundos estoura: o mesmo `502 QIT001019`.
4. Nos três casos a falha acontece antes da trava da estante e antes da gravação: nada é travado e nada é gravado.

Mover um livro de estante — caminho feliz



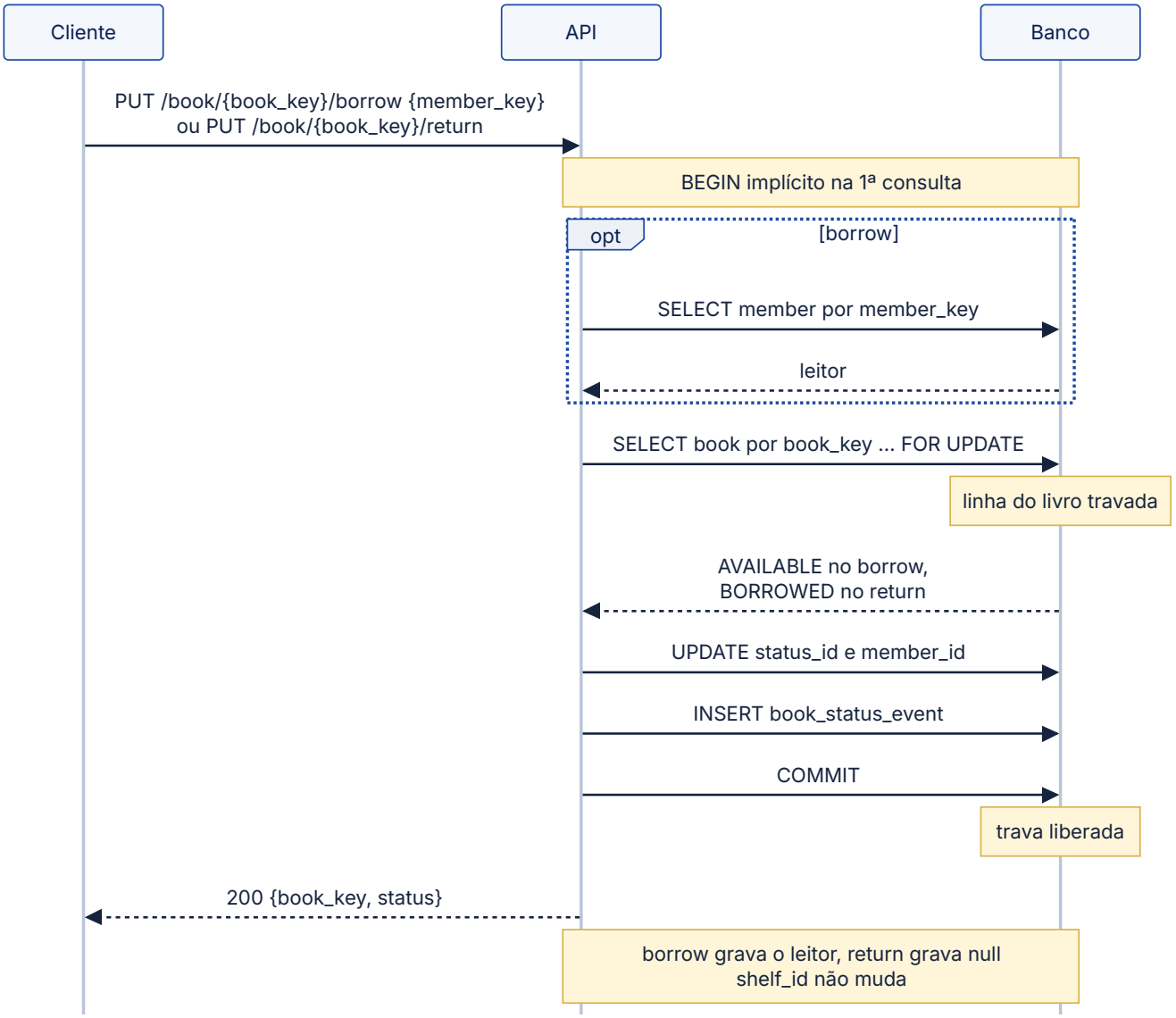
1. Procura o livro pela `book_key` e a estante pela `shelf_key`. Sem livro, `404 QIT001015`; sem estante, `404 QIT001013`.
2. Se o livro já está nessa estante, responde `409 QIT001025` antes de travar ou contar: o pedido não moveria nada, e quem chamou precisa saber disso. A conferência vem antes da trava também porque, numa estante cheia, o próprio livro entraria na conta e a resposta sairia como estante lotada, que é o motivo errado.
3. Trava a linha da estante de destino (`SELECT … FOR UPDATE`) e conta os livros dela. Livro `BORROWED` continua na estante e entra na conta: a vaga fica reservada para quando ele voltar.
4. Cabe: grava o novo `shelf_id`, confirma a transação (a trava é liberada) e responde `200` com `book_key` e `status`.

Mover um livro de estante — falha: dois pedidos disputam a última vaga



1. A estante tem capacidade 10 e 9 livros. As requisições A e B pedem para pôr um livro nela.
2. A trava a linha da estante e conta 9. B pede a mesma trava e fica parada no banco até A terminar.
3. A grava e confirma a transação. A estante passa a ter 10 livros e a trava é liberada.
4. B recebe a trava, conta 10 e responde `409 QIT001017` com o código e a capacidade da estante. A transação de B termina sem gravar nada.

Emprestar e devolver — caminho feliz

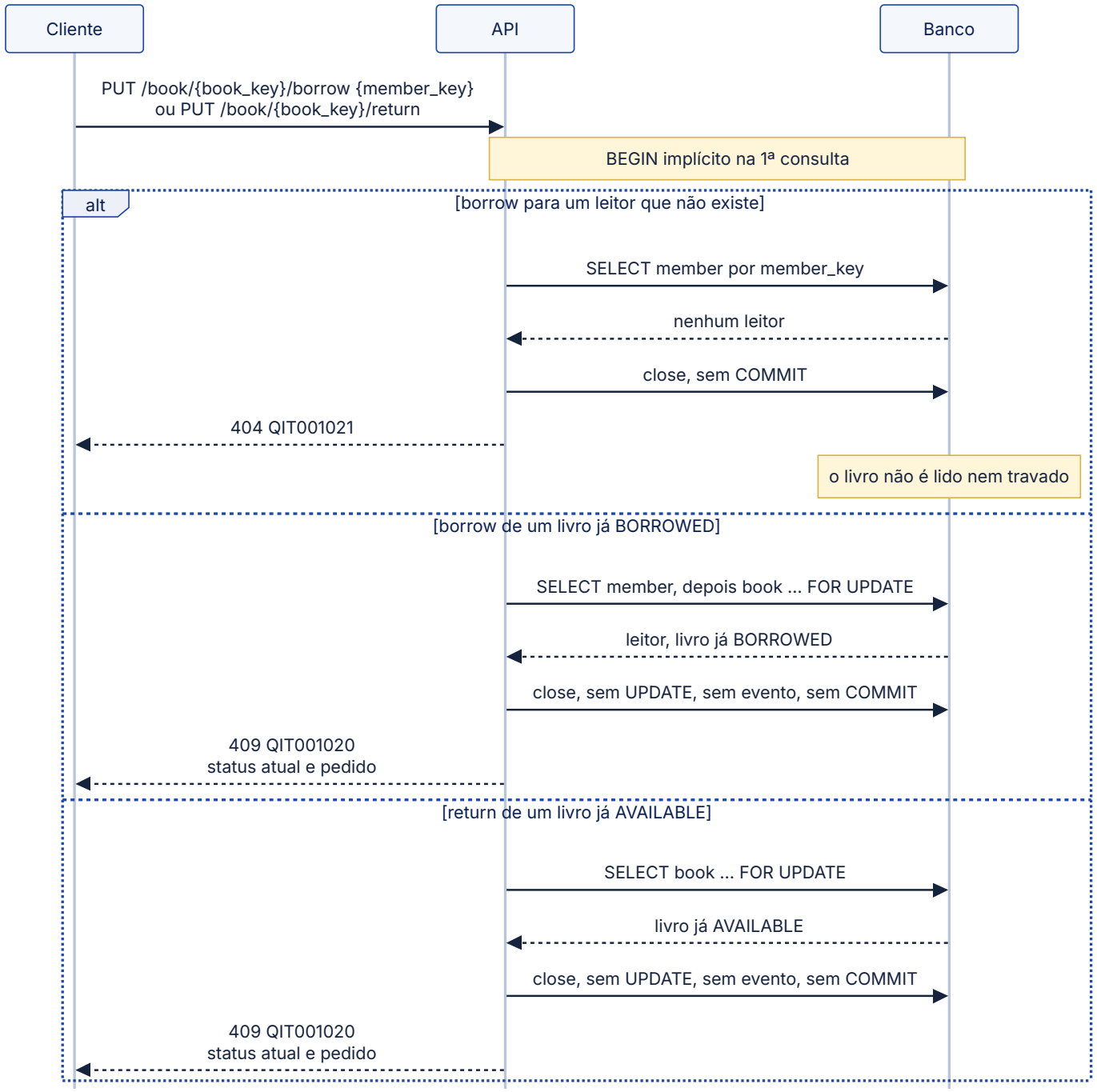


1. No empréstimo, procura o leitor pela `member_key` do corpo. Sem leitor, `404 QIT001021`, antes de o livro ser lido. Corpo sem `member_key` sai antes, com `400 QIT000001`. A devolução não tem corpo e não passa por aqui.
2. Procura o livro pela `book_key` e trava a linha dele (`SELECT … FOR UPDATE`). Sem livro, `404 QIT001015`.
3. Emprestar parte de `AVAILABLE`; devolver parte de `BORROWED`. O livro está no status de partida.
4. Grava o status de chegada e o leitor juntos: o empréstimo grava o leitor do corpo em `member_id`, a devolução grava `null`, e os dois gravam um evento com o status de chegada em `book_status_event`. Confirma a transação, o que libera a trava, e responde `200` com `book_key` e `status`. O livro não sai da estante em nenhum dos dois sentidos.

A trava no livro evita a corrida de dois empréstimos do mesmo livro ao mesmo tempo. Sem ela, as duas requisições leem `AVAILABLE`, as duas gravam `BORROWED` com o próprio leitor e as duas recebem `200`; o livro fica no nome de quem gravou por último, que pode não ser quem saiu com ele. A regra "emprestado tem leitor; disponível não tem" não pega esse caso, porque `BORROWED` com leitor é um estado válido: o que está errado é qual leitor. Essa

regra vale no controller, dentro da transação, porque um `CHECK` em `book` não enxerga o `enumerator`, que mora em `book_status`. Com o `SELECT … FOR UPDATE`, a segunda requisição espera o `commit` da primeira, lê `BORROWED` e recebe `409 QIT001020`. A devolução usa a mesma trava, para que um empréstimo e uma devolução cruzados não leiam o mesmo estado.

Emprestar e devolver — falha: leitor inexistente ou transição inválida



1. Pedem para emprestar a um `member_key` que não existe: `404 QIT001021`. O leitor é conferido antes da trava, e a linha do livro não chega a ser travada.
2. Pedem para emprestar um livro que já está `BORROWED`, ou para devolver um que está `AVAILABLE`.
3. A API responde `409 QIT001020`, dizendo o status atual e o que foi pedido. Nada é gravado, nenhum evento entra em `book_status_event`, e o livro

continua como estava, com o mesmo leitor ou sem nenhum.

Principal desafio

- **Qual é:** garantir que uma estante nunca passe da capacidade, com dois pedidos disputando a última vaga ao mesmo tempo — no cadastro de um livro com `shelf_key` e na troca de estante.
- **Por que é difícil:** a solução óbvia — contar os livros e, se couber, gravar — tem um intervalo entre a contagem e a gravação, e nesse intervalo outra requisição conta o mesmo número. As duas acertam a conta e as duas gravam. No cadastro há uma tensão a mais: travar cedo demais seguraria a estante durante a chamada ao catálogo, até 5 segundos, e pararia todo pedido daquela estante.
- **Como o desenho resolve:** a linha da estante é travada com `SELECT … FOR UPDATE` antes de contar, e a contagem e a gravação do livro acontecem na mesma transação; a trava só é liberada no `commit`, depois da gravação. Quem chega depois espera a trava, conta o número novo e recebe `409` `QIT001017`. No cadastro, a trava é a última coisa antes de gravar, depois da resposta do catálogo, e por isso nenhuma linha fica travada durante a chamada HTTP. Uma leitura comum da estante (`GET`) não pede a trava e não espera.